

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development
Course Code	23CM4121
TASK NO.	Task6(PBL Q.no: 6)
Title	Node.js Modules – os, path and fs
Student Name	Ch. Venu Gopal Reddy
Roll No.	A24126552078
College	Anil Neerukonda Institute of Technology and Sciences (ANITS)
Date	_____

1. TITLE

Node.js Modules – Demonstration of os, path and fs Built-in Modules

2. AIM

To understand Node.js modules and demonstrate the purpose and usage of the built-in os, path, and fs modules through a simple Node.js program.

3. OBJECTIVE

- To understand the concept of modules in Node.js.
- To use the built-in os module to obtain operating-system and hardware information.
- To use the path module to create and analyze file and directory paths.
- To use the fs (File System) module to create, write, read, append, and check files.
- To execute and verify the program using the Node.js terminal.

4. THEORY

Node.js supports a modular programming approach. A module is a reusable unit of code that provides related functionality. Node.js includes several built-in modules that can be imported using `require()`. This experiment demonstrates three commonly used built-in modules: os, path, and fs.

The os module provides information about the operating system and computer hardware, such as the platform, operating-system type, CPU architecture, number of CPU cores, memory information, and home directory.

The path module provides utilities for working with file and directory paths. It can join path components and extract information such as the directory name, file name, and file extension.

The fs module provides file-system operations. It can be used to create files, write data, read data, append data, and check whether a file exists. In this program, synchronous methods are used so that the execution steps can be observed clearly in sequence.

5. ALGORITHM / PROCEDURE

1. Create a Node.js project folder and create a JavaScript file named `node_modules_demo.js`.
2. Import the built-in os, path, and fs modules using `require()`.
3. Use the os module to display operating-system, CPU, memory, and home-directory information.
4. Use the path module to join a folder name and file name and display the directory name, base name, and extension.
5. Use the fs module to create a `student_data.txt` file and write initial student information into it.
6. Read and display the file contents using the fs module.
7. Append additional information to the file and read the updated contents.
8. Check whether the file exists using `fs.existsSync()`.
9. Run the program using `node node_modules_demo.js` and verify the output for all operations.

6. PROGRAM

```
// Q6: Node.js Modules Demonstration
// Demonstrating built-in modules: os, path, and fs

// Importing the required Node.js built-in modules
const os = require("os");
const path = require("path");
const fs = require("fs");

// =====
// PART 1: OS MODULE
// =====

console.log("===== OS MODULE =====");

console.log("Operating System:", os.platform());
console.log("OS Type:", os.type());
console.log("CPU Architecture:", os.arch());
console.log("Number of CPU Cores:", os.cpus().length);
console.log("Total Memory:", (os.totalmem() / (1024 ** 3)).toFixed(2), "GB");
console.log("Free Memory:", (os.freemem() / (1024 ** 3)).toFixed(2), "GB");
console.log("Home Directory:", os.homedir());

// =====
// PART 2: PATH MODULE
// =====

console.log("\n===== PATH MODULE =====");

const folder = "documents";
const fileName = "student.txt";

const fullPath = path.join(folder, fileName);

console.log("Folder:", folder);
console.log("File Name:", fileName);
console.log("Complete Path:", fullPath);
console.log("Directory Name:", path.dirname(fullPath));
console.log("File Name:", path.basename(fullPath));
console.log("File Extension:", path.extname(fullPath));

// =====
// PART 3: FS MODULE
// =====

console.log("\n===== FS MODULE =====");

const filePath = path.join(__dirname, "student_data.txt");

const initialData =
```

```

    "Name: Venu\n" +
    "Roll Number: A24126552078\n" +
    "Branch: CSM\n";

fs.writeFileSync(filePath, initialData);
console.log("File created and data written successfully.");

const fileData = fs.readFileSync(filePath, "utf8");
console.log("\nData read from file:");
console.log(fileData);

const additionalData = "College: ANITS\n";
fs.appendFileSync(filePath, additionalData);
console.log("Additional data appended successfully.");

const updatedData = fs.readFileSync(filePath, "utf8");
console.log("\nUpdated file content:");
console.log(updatedData);

if (fs.existsSync(filePath)) {
    console.log("File exists:", filePath);
}

console.log("\n===== PROGRAM COMPLETED =====");
console.log("OS, Path and FS modules demonstrated successfully.");

```

7. CODE EXPLANATION

S.No.	Code / Function	Explanation
1	const os = require("os")	Imports the os module.
2	const path = require("path")	Imports the path module.
3	const fs = require("fs")	Imports the File System module.
4	os.platform(), os.type(), os.arch()	Displays OS platform, type and CPU architecture.
5	os.cpus().length	Displays the number of CPU cores.
6	os.totalmem(), os.freemem()	Displays total and free system memory.
7	path.join(folder, fileName)	Combines path components into a path.
8	path.dirname(), path.basename(), path.extname()	Extracts directory, file name and extension.
9	fs.writeFileSync()	Creates/writes the student_data.txt

file.

10	<code>fs.readFileSync()</code>	Reads and displays file content.
11	<code>fs.appendFileSync()</code>	Adds data to the existing file.
12	<code>fs.existsSync()</code>	Checks whether the file exists.

MODULES USED

Module	Purpose	Example Used
os	Operating-system and hardware information.	<code>os.platform()</code> , <code>os.cpus()</code> , <code>os.totalmem()</code>
path	File and directory path handling.	<code>path.join()</code> , <code>path.basename()</code> , <code>path.extname()</code>
fs	File-system operations.	<code>writeFileSync()</code> , <code>readFileSync()</code> , <code>appendFileSync()</code>

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run <code>node_modules_demo.js</code>	Program starts without errors	OS MODULE heading and OS information displayed	PASS
TC-02	Execute os module	OS information displayed	Platform, type and architecture displayed	PASS
TC-03	Execute CPU/memory section	CPU and memory values displayed	CPU count, total memory and free memory displayed	PASS
TC-04	Execute path module	Path components displayed	Complete path, directory, file name and .txt extension displayed	PASS
TC-05	Execute write operation	<code>student_data.txt</code> created	File created and data written successfully	PASS
TC-06	Execute read operation	Initial data displayed	Name, roll number and branch	PASS

			displayed	
TC-07	Execute append operation	New data added	College: ANITS displayed in updated content	PASS
TC-08	Execute exists check	File existence confirmed	File exists message displayed	PASS
TC-09	Complete execution	All modules execute successfully	PROGRAM COMPLETED message displayed	PASS

Note: A single terminal screenshot showing the complete successful execution can be used as evidence for multiple test cases, provided the relevant outputs are clearly visible.

9. OUTPUT

OUTPUT SCREENSHOT – OS MODULE

```

===== OS MODULE =====
Operating System: win32
OS Type: Windows_NT
CPU Architecture: x64
Number of CPU Cores: 12
Total Memory: 7.57 GB
Free Memory: 0.86 GB
Home Directory: C:\Users\venug

```

Paste a screenshot showing OS platform, type, architecture, CPU and memory information.

OUTPUT SCREENSHOT – PATH MODULE

```

===== PATH MODULE =====
Folder: documents
File Name: student.txt
Complete Path: documents\student.txt
Directory Name: documents
File Name: student.txt
File Extension: .txt

```

Paste a screenshot showing the complete path, directory name, file name and extension.

OUTPUT SCREENSHOT – FS MODULE / MULTIPLE TEST CASES

```
===== FS MODULE =====
File created and data written successfully.

Data read from file:
Name: Venu
Roll Number: A24126552078
Branch: CSM

Additional data appended successfully.

Updated file content:
Name: Venu
Roll Number: A24126552078
Branch: CSM
College: ANITS

File exists: F:\full-stack-web-development\01-Observation\Task6-PBLpart2\6_node_modules\student_data.txt
```

Paste a terminal screenshot showing file creation, reading, appending, file existence and the final completion message. One screenshot may cover multiple passed test cases.

10. RESULT

Thus, the Node.js built-in modules `os`, `path`, and `fs` were successfully studied and demonstrated. The program retrieved operating-system information, processed file paths, created and modified a file, read its contents, and verified file existence successfully.

11. SUBMISSION CONTENT

- Source file: `node_modules_demo.js`
- Generated file during execution: `student_data.txt`
- Terminal output screenshots showing successful test-case execution
- Completed laboratory observation / record